Experiments

1. Overview

This document summarizes the ISA-level experiments for accelerating a quantized DeiT-Tiny Transformer workload on bare-metal RV64IMC.

The current evaluation focuses on:

- W8A8 quantized inference
- Bare-metal RV64IMC execution
- Spike ISA-level profiling
- `Xqmac8` packed INT8 MAC instruction
- `Xrequant` fixed-point requantization instruction
- Operator-level evaluation
- Complete DeiT-Tiny Transformer Block 0 evaluation

The current results are **ISA-level results**. Spike is used to measure dynamic instruction behavior and functional correctness. Cycle-accurate CV-Wally RTL evaluation, CPI, Fmax, and area are not included in this midterm-stage evaluation.

---

2. Experimental Setup

2.1 Model

Primary model:

```
Model       : DeiT-Tiny
Parameters  : approximately 5.7 M
Compute     : approximately 1.3 GMAC / inference
Embedding   : 192
Heads       : 3
Head dim    : 64
Blocks      : 12
Tokens      : 197
```

The complete Transformer Block 0 used in the integrated experiment is:

```
X0
 |
 v
LayerNorm 1
 |
 v
QKV Projection
 |
```

```
 +----------------+
 |                |
 v                |
QK^T              | V
 |                |
 v                |
Integer Softmax   |
 |                |
 +-------+--------+
         |
         v
 Attention x V
         |
         v
 Output Projection
         |
         v
 Residual Add 1
         |
         v
 LayerNorm 2
         |
         v
        FC1
         |
         v
     GELU LUT
         |
         v
        FC2
         |
         v
 Residual Add 2
         |
         v
 Block 0 Output
```

2.2 Quantization

The current implementation uses symmetric W8A8 quantization.

```
Weights       : INT8
Activations   : INT8
Zero point    : 0
Accumulator   : INT32
```

Activation quantization:

```
per-tensor symmetric
```

Weight quantization:

```
per-output-channel symmetric
```

For an output channel j:

$$s_{acc,j} = s_x s_{w,j}$$

and requantization uses:

$$M_j = \frac{s_x s_{w,j}}{s_y}$$

The multiplier is represented in Q31 format:

$$M_{int,j} = \mathrm{round}(M_j 2^{31})$$

The implemented rounding policy is:

```
round to nearest
ties away from zero
```

The final INT8 range is:

$$[-127, 127]$$

---

3. ISA Configurations

The following configurations are used throughout the experiments.

Configuration	Description
B0	Baseline scalar RV64IMC
B1	Software-optimized RV64IMC
X1	RV64IMC + `Xqmac8`
X2	RV64IMC + `Xqmac8` + `Xrequant`

---

4. Custom Instructions

4.1 Xqmac8

`Xqmac8` performs eight signed INT8 multiply-accumulate operations.

Conceptually:

$$rd \leftarrow int32\left(rd + \sum_{i=0}^{7} int8(rs1_i) \times int8(rs2_i)\right)$$

Encoding:

```
Opcode : CUSTOM-0 = 0x0B
funct3 : 000
funct7 : 0000000
```

Spike encoding:

```
MATCH_XQMAC8 = 0x0000000b
MASK_XQMAC8  = 0xfe00707f
```

The instruction uses:

```
rd  : INT32 accumulator, source and destination
rs1 : packed 8 x INT8 operand
rs2 : packed 8 x INT8 operand
```

The result follows INT32 wrap semantics and is sign-extended to RV64.

4.2 Xrequant

`Xrequant` converts an INT32 accumulator to the output INT8 quantization domain.

Conceptually:

```
INT32 accumulator
        |
        v
signed INT64 product with Q31 multiplier
        |
        v
round to nearest, ties away from zero
        |
        v
shift right by 31
        |
        v
saturate to [-127, 127]
```

Encoding:

```
Opcode : CUSTOM-0 = 0x0B
funct3 : 001
funct7 : 0000000
```

Spike encoding:

```
MATCH_XREQUANT = 0x0000100b
MASK_XREQUANT  = 0xfe00707f
```

The current instruction is specialized for the present symmetric W8A8 workload:

```
zero point  = 0
fixed shift = 31
```

It should therefore not be interpreted as a fully general-purpose requantization instruction.

---

5. Functional Verification

5.1 Directed ISA Tests

Both custom instructions passed directed functional tests.

Instruction	Result
`Xqmac8`	PASS
`Xrequant`	PASS

5.2 Randomized ISA Tests

Randomized testing was performed using a fixed random seed.

Instruction	Test cases	Result
`Xqmac8`	10,000	PASS
`Xrequant`	10,000	PASS

5.3 Operator Verification

The following quantized kernels have been compared against their software references.

Operator	Result
FC1	PASS

Operator	Result
FC2	PASS
QKV	PASS
QK^T	PASS
Integer Softmax	PASS
Attention x V	PASS
Attention Output Projection	PASS
Connected MLP	PASS

5.4 Complete Transformer Block Verification

The complete connected Transformer Block 0 was executed using:

```
Integer LayerNorm
QKV
QK^T
Integer Softmax
Attention x V
Projection
Residual 1
Integer LayerNorm
FC1
GELU LUT
FC2
Residual 2
```

The bare-metal RV64IMC + X2 implementation matched the Python integer reference exactly.

```
Block0 X2 verify = 0
```

The final Block0 output contains:

$$197 \times 192 = 37,824$$

INT8 output elements.

All 37,824 final output elements matched the Python integer reference bit-exactly.

Therefore:

$$\boxed{\text{Python Integer Block0} = \text{Spike X2 Block0}}$$

5.5 Floating-Point Instruction Check

The final Block0 executable was disassembled and checked for floating-point instructions.

The following instruction classes were searched:

```
fadd
fsub
fmul
fdiv
fsqrt
flw
fld
fsw
fsd
```

No unintended floating-point instructions were found.

---

6. Quantization Quality

Functional bit-exactness and quantization accuracy are treated separately.

The comparison is:

```
FP32 Block0
     |
     | quantization error
     v
Python Integer Block0
     |
     | implementation correctness
     v
Spike X2 Block0
```

Therefore:

```
FP32 != Integer
```

is expected due to quantization, while:

```
Python Integer == Spike Integer
```

is required for implementation correctness.

6.1 Connected Block0 Error

Measured against the FP32 Block0 output:

Metric	Value
LN1 MAE	0.028309
Residual1 MAE	0.064121
LN2 MAE	0.088567
Final Block0 MAE	**0.078367**
Final max error	**0.438896**
Final SQNR	**19.63 dB**

The connected integer implementation therefore remains numerically stable across the complete Transformer block.

This result should not be interpreted as full ImageNet classification accuracy. Full-model classification accuracy has not yet been evaluated.

---

7. Workload Compute Breakdown

The complete Block0 contains the following useful MAC operations.

Operator	Useful MACs	Share
QKV	21,786,624	21.35%
QK^T	7,451,328	7.30%
Attention x V	7,451,328	7.30%
Projection	7,262,208	7.12%
FC1	29,048,832	28.47%
FC2	29,048,832	28.47%
Total	**102,049,152**	**100%**

The MLP portion:

$$FC1 + FC2$$

accounts for:

$$\boxed{56.93\%}$$

of Block0 useful MAC operations.

7.1 AV Padding

Attention x V has an inner dimension of 197.

For `Xqmac8`, the inner dimension is padded to:

$$200$$

so that it is divisible by 8.

Useful AV MACs:

$$3 \times 197 \times 64 \times 197 = 7,451,328$$

Executed padded AV MACs:

$$3 \times 197 \times 64 \times 200 = 7,564,800$$

Padding overhead:

$$\boxed{\approx 1.52\%}$$

Therefore:

```
Useful Block0 MACs   = 102,049,152
Executed Block0 MACs = 102,162,624
```

The normalized metrics in this document use **useful MACs** unless otherwise stated.

---

8. Microbenchmark Evaluation

A fair dot-product microbenchmark was used to compare scalar loads against packed `Xqmac8`.

Workload:

```
1,000,000 dot8 operations
8,000,000 MACs
```

Results:

Configuration	Dynamic instructions
Scalar RV64IMC	34,000,054
Xqmac8	5,000,015

Instruction-count ratio:

$$\frac{34,000,054}{5,000,015} \approx \boxed{6.80\times}$$

Dynamic instruction reduction:

$$\boxed{85.29\%}$$

Loads per MAC:

```
Scalar : 2.00
Xqmac8 : 0.25
```

MAC per instruction:

```
Scalar : 0.235
Xqmac8 : 1.600
```

This experiment demonstrates the direct effect of packing eight INT8 MAC operations into one custom instruction.

---

9. Major Operator Evaluation

9.1 FC1

Shape:

$$[197 \times 192] \times [192 \times 768] \rightarrow [197 \times 768]$$

MACs:

$$29,048,832$$

Dynamic instruction results:

Configuration	Main instructions	Ratio vs B0
B0	206,369,535	1.00x
B1	130,419,162	1.58x
X1	24,965,631	8.27x
X2	23,603,960	**8.74x**

Incremental improvements:

$$B0/B1 = 1.58\times$$

$$B1/X1 = 5.22\times$$

$$X1/X2 = 1.058\times$$

Overall B0 -> X2 dynamic instruction reduction:

$$\boxed{88.56\%}$$

X1 custom count:

```
Xqmac8 = 3,631,104
```

X2 custom counts:

```
Xqmac8   = 3,631,104
Xrequant =   151,296
```

9.2 FC2

Shape:

$$[197 \times 768] \times [768 \times 192] \rightarrow [197 \times 192]$$

MACs:

$$29,048,832$$

Dynamic instruction results:

Configuration	Main instructions	Ratio vs B0
B0	204,100,095	1.00x
B1	127,922,778	1.60x
X1	22,582,719	9.04x
X2	22,242,296	**9.18x**

Overall B0 -> X2 reduction:

$$\boxed{89.10\%}$$

The smaller improvement from X1 to X2 compared with FC1 is expected because FC2 has fewer output elements and therefore fewer requantization operations.

9.3 QKV

Shape:

$$[197 \times 192] \times [192 \times 576] \rightarrow [197 \times 576]$$

MACs:

$$21,786,624$$

Results:

Configuration	Main instructions
B0	154,777,599

Configuration	Main instructions
X2	20,313,272

Instruction-count ratio:

$$\boxed{7.62\times}$$

Reduction:

$$\boxed{86.88\%}$$

Theoretical `Xqmac8` executions:

$$\frac{21,786,624}{8} = 2,723,328$$

Requantization outputs:

$$197 \times 576 = 113,472$$

9.4 QK^T

For each attention head:

$$[197 \times 64] \times [64 \times 197] \rightarrow [197 \times 197]$$

Across three heads:

$$7,451,328$$

MACs.

Results:

Configuration	Main instructions
B0	60,544,442
X1	8,268,733

Instruction-count ratio:

$$\boxed{7.32\times}$$

Reduction:

$$\boxed{86.34\%}$$

B0:

```
Instructions/MAC = 8.125
MAC/Instruction  = 0.123
Loads/MAC        = 2.000
```

X1:

```
Instructions/MAC = 1.110
MAC/Instruction  = 0.901
Loads/MAC        = 0.250
```

Theoretical and measured custom instruction count:

$$\frac{7,451,328}{8} = \boxed{931,416}$$

The measured `.insn` count was also 931,416.

---

10. Complete Block0 ISA Evaluation

10.1 Dynamic Instruction Count

Useful MACs:

$$102,049,152$$

Results:

Metric	B0	X2
Program instructions	743,784,995	105,759,995
Main instructions	740,886,472	102,863,474

Using total program instructions:

$$\frac{743,784,995}{105,759,995} = \boxed{7.03\times}$$

Dynamic instruction reduction:

$$\boxed{85.78\%}$$

Using the main workload region:

$$\frac{740,886,472}{102,863,474} = \boxed{7.20\times}$$

Reduction:

$$\boxed{86.12\%}$$

These values represent an **ISA-level dynamic-instruction reduction**, not a cycle-accurate hardware speedup.

10.2 Derived Metrics

Metric	B0	X2
Instructions / MAC	7.260094	**1.007980**
MAC / instruction	0.137739	**0.992083**
Loads / MAC	2.018695	**0.266749**
Stores / MAC	0.008652	0.008652
Multiply / MAC	1.006301	**0**

Loads/MAC improve by:

$$\frac{2.018695}{0.266749} \approx \boxed{7.57\times}$$

---

11. Complete Block0 Instruction Mix

11.1 B0

Category	Count	Share
Load	206,006,072	27.81%
Store	882,953	0.12%
Multiply	102,692,160	13.86%
Branch/Jump	105,138,531	14.19%
Integer ALU/Other	326,166,756	44.02%

Dominant mnemonics:

Mnemonic	Count	Share
`addi`	206,858,503	27.92%
`lb`	204,476,544	27.60%
`bne`	103,086,934	13.91%
`addw`	102,162,819	13.79%
`mulw`	102,162,624	13.79%

This shows the cost of the scalar MAC loop:

```
load
load
```

```
multiply
accumulate
address update
branch
```

11.2 X2

Category	Count	Share
Load	27,221,480	26.46%
Store	882,953	0.86%
Multiply	**0**	**0.00%**
Custom	13,299,864	12.93%
Branch/Jump	14,542,124	14.14%
Integer ALU/Other	46,917,053	45.61%

Dominant mnemonics:

Mnemonic	Count	Share
`addi`	28,073,911	27.29%
`ld`	25,885,820	25.17%
`bne`	13,769,695	13.39%
`.insn`	13,299,864	12.93%
`sext.w`	13,224,729	12.86%

The scalar multiply bottleneck has therefore been eliminated, exposing other overheads.

---

12. Analytical Validation of Custom Instruction Utilization

12.1 Xqmac8

Expected dynamic `Xqmac8` counts:

Operator	Xqmac8 count
QKV	2,723,328
QK^T	931,416
Attention x V	945,600
Projection	907,776
FC1	3,631,104

Operator	Xqmac8 count
FC2	3,631,104
Total	**12,770,328**

12.2 Xrequant

Expected `Xrequant` counts:

Stage	Xrequant count
QKV	113,472
Attention x V	37,824
Projection	37,824
Residual 1	75,648
FC1	151,296
FC2	37,824
Residual 2	75,648
Total	**529,536**

Therefore:

$$12,770,328 + 529,536 = 13,299,864$$

Measured Spike result:

```
Custom = 13,299,864
.insn  = 13,299,864
```

Thus:

$$\boxed{\text{Measured custom instructions} = \text{Analytically expected custom instructions}}$$

---

13. Code Size

Complete Block0 `.text` sizes:

Configuration	`.text` size
B0	2748 B
X2	2164 B

Reduction:

$$2748 - 2164 = 584\ \mathrm{B}$$

Percentage reduction:

$$\boxed{21.25\%}$$

Only the `.text` section is used for code-size comparison because static quantized model parameters dominate the ELF data size.

---

14. Bottleneck Analysis

14.1 Baseline Bottleneck: Scalar MAC Expansion

The baseline requires separate instructions for:

```
activation load
weight load
multiply
accumulate
pointer update
loop branch
```

This results in:

$$7.260$$

instructions per useful MAC for the complete Block0.

The packed `Xqmac8` instruction directly addresses this bottleneck.

14.2 Baseline Bottleneck: Load Overhead

B0 requires:

$$2.019$$

loads per useful MAC.

After packed MAC:

$$0.267$$

loads per useful MAC remain.

Thus `Xqmac8` substantially improves data reuse at the ISA level by allowing one 64-bit load to provide eight INT8 elements.

14.3 Baseline Bottleneck: Requantization

Scalar requantization requires fixed-point multiplication, sign handling, rounding, shifting, and saturation.

The complete Block0 executes:

$$529,536$$

requantization operations.

`Xrequant` fuses these steps into one custom instruction.

The effect is especially visible in operators with a large number of output elements, such as FC1.

---

15. Bottlenecks After X2

After `Xqmac8` and `Xrequant`, multiplication is no longer a dominant instruction class.

The dominant X2 instructions are:

```
addi      27.29%
ld        25.17%
bne       13.39%
.insn     12.93%
sext.w    12.86%
```

This reveals the next architectural bottlenecks.

15.1 Data Movement

Loads still represent:

$$\boxed{26.46\%}$$

of X2 main instructions.

Potential future directions include:

- fused load + MAC
- limited dual issue
- improved cache organization
- software-managed scratchpad

15.2 Address Generation and Loop Control

`addi` represents:

$$\boxed{27.29\%}$$

while branch/jump instructions represent:

$$\boxed{14.14\%}$$

This suggests that pointer updates and loop control become significant once arithmetic is compressed.

Potential future directions include:

- post-increment load/store
- hardware loop support

15.3 RV64 Sign-Extension Overhead

`sext.w` represents:

$$\boxed{12.86\%}$$

of X2 main instructions.

This is largely related to the INT32 accumulator representation on RV64 and the compiler-visible custom-instruction interface.

It should currently be considered a software/compiler interface overhead rather than direct evidence that an additional hardware instruction is necessary.

---

16. Main Findings

The current experiments support the following conclusions.

1. Scalar INT8 MAC expansion is a major instruction-count bottleneck on RV64IMC.
2. `Xqmac8` significantly reduces arithmetic, load, and loop overhead by processing eight INT8 MACs per custom instruction.
3. `Xrequant` further reduces output-stage fixed-point conversion overhead.
4. The complete connected Transformer Block0 achieves:

$$\boxed{86.12\%}$$

main dynamic-instruction reduction.

5. The instruction-count ratio between B0 and X2 is:

$$\boxed{7.20\times}$$

for the main Block0 workload.

6. Total program instruction count decreases by:

$$\boxed{85.78\%}$$

or a:

$$\boxed{7.03\times}$$

instruction-count ratio.

7. Complete Block0 INT8 output is bit-exact between the Python integer reference and Spike.

8. No unintended floating-point instruction is required.

9. After packed compute optimization, the dominant remaining costs become:

```
data movement
address generation
loop control
RV64 sign-extension/compiler overhead
```

These observations provide the motivation for future microarchitectural extensions.

---

17. Reproducibility

Main repository checkpoint:

```
Commit:
5ebd201
midterm: complete quantized DeiT Block0 ISA evaluation

Tag:
midterm-block0-v1
```

The repository later adds the Spike patch at:

```
Commit:
f507d92
repo: add Spike Xqmac8 and Xrequant patch
```

Spike base commit:

```
0bff121
```

Custom Spike patch:

```
patches/spike-xqmac-xrequant.patch
```

The patch was verified using:

```
git apply --check
```

against a clean Spike worktree based on commit `0bff121`.

Important result files include:

```
results/dot8_fair_profile.txt

results/fc2_b0_profile.txt
results/fc2_b1_profile.txt
results/fc2_x1_profile.txt
results/fc2_x2_profile.txt

results/qkv_b0_profile.txt
results/qkv_x2_profile.txt

results/qkt_b0_profile.txt
results/qkt_x1_profile.txt

results/block0_b0_profile.txt
results/block0_x2_profile.txt
```

The complete Block0 benchmark source is:

```
src/block0_bench.c
```

and complete X2 functional verification is:

```
src/block0_x2_verify.c
```

The complete generated static model data is stored in:

```
src/generated/block0_data.c
src/generated/block0_data.h
```

---

18. Current Scope and Future Work

The current midterm-stage work focuses on:

```
ISA design
Spike implementation
functional verification
dynamic instruction analysis
complete Transformer Block evaluation
```

The following are intentionally deferred to later project stages:

- CV-Wally RTL integration
- cycle-accurate evaluation
- CPI measurement
- Fmax / critical path
- synthesis area
- performance / area tradeoff
- full 12-block DeiT-Tiny inference
- ImageNet classification accuracy

The immediate architectural question for the next stage is:

> After removing most scalar MAC and requantization overhead, which memory, pipeline, or loop-control extension provides the best additional performance improvement for the observed X2 bottlenecks?